

Memoria del Proyecto

Grupo 10

Juan Andrés Hibjan Cardona

Índice de contenidos

1. Análisis del sistema.....	1
1.1 Contrato <i>QuadraticVoting</i>	1
Funciones.....	1
Modificadores y elección entre modifier y require.....	4
Vulnerabilidades consideradas y defensa.....	4
1.2 Contrato <i>VotingToken</i>	5
1.3 <i>IExecutableProposal</i> y <i>TestProposal</i>	6
2. Pull-over-push.....	6
3. Casos de prueba.....	7
1. Despliegue de <i>TestProposal</i> y <i>QuadraticVoting</i>	7
2. Inscripción de participantes antes de abrir la votación.....	8
3. Apertura de la votación.....	10
4. Creación de propuestas.....	10
5. Voto cuadrático y aprobación automática.....	12
6. Retirada parcial de votos.....	15
7. Cancelación.....	16
8. Cierre de la votación y flujos pull.....	17
9. Reapertura.....	19
10. Comprobaciones de seguridad.....	20

1. Análisis del sistema

1.1 Contrato *QuadraticVoting*

Mantiene los siguientes campos de estado:

owner, votingToken, tokenPrice: declarados como immutable, se fijan en el constructor y se incrustan en el bytecode (ahorran SLOAD en cada lectura).

votingOpen: flag de la ronda en curso.

totalBudget: presupuesto vivo expresado en wei.

participantCount, proposalCount: contadores que solo se incrementan.

currentVotingRound: número de ronda activa, sirve de clave para los arrays por ronda.

participants: mapping(address => bool).

proposals: mapping(uint => Proposal).

pendingFinancingIds, approvedProposalIds, signalingProposalIds: indexados por ronda (mapping(uint => uint[])).

roundClosed: mapping(uint => bool), habilita los flujos pull tras closeVoting.

La estructura **Proposal** almacena título, descripción, budget, dirección del contrato externo, creador, estado, totalVotes, totalTokensStaked, arrayIndex (posición en su array de ronda para el swap and pop), votingRound, signalingExecuted (para evitar doble ejecución de signaling) y voterVotes.

Funciones

openVoting() external payable onlyOwner onlyVotingClosed

Coste $O(1)$. Es external porque jamás se invoca internamente; payable para recibir el presupuesto inicial; onlyOwner porque solo el creador del contrato puede abrir periodo, y onlyVotingClosed para impedir reaperturas. El incremento de currentVotingRound se ejecuta en bloque unchecked porque alcanzar 2^{256} es, en la práctica, imposible en este contador. No se borra el estado de rondas anteriores: como los arrays están en mapping por ronda, simplemente se accede a una clave nueva. Esta decisión se ha tomado para conseguir un consumo de gas constante.

closeVoting() external onlyOwner onlyVotingOpen

Coste $O(1)$ (clave del rediseño pull over push). Pone votingOpen a false, marca roundClosed[currentVotingRound] y transfiere el presupuesto sobrante al owner mediante una llamada de bajo nivel (call de bajo nivel, requerido porque el owner puede ser un contrato y send/transfer aplicarían un techo de 2 300 unidades de gas que rompería la compatibilidad). Sigue el patrón checks effects interactions: totalBudget se pone a cero antes del call, neutralizando reentradas en este punto.

addParticipant() external payable

Coste $O(1)$. Es external payable porque exige al menos tanto ETH como el valor de un token. No exige onlyVotingOpen ya que la inscripción debe poder hacerse en cualquier momento, según los

requisitos. Comprueba que el llamante no esté ya inscrito, marca el flag, incrementa participantCount (unchecked, justificado por la naturaleza monotónica del contador) y delega la compra de tokens en _buyTokens. La protección frente a reentradas viene del orden de operaciones (effects antes que interactions): cuando _buyTokens hace el call para devolver el cambio, la cuenta ya consta como participante, así que una reentrada a addParticipant fallaría en el require inicial.

removeParticipant() external onlyParticipant

Coste O(1). El modificador onlyParticipant garantiza participantCount >= 1, lo que permite usar unchecked en la decrementación. No quema tokens ni reembolsa votos: los tokens permanecen en propiedad del usuario y los votos depositados siguen contando hasta que la ronda se cierre y se reclamen vía claimRefund.

buyTokens() external payable onlyParticipant

Coste O(1). Wrapper sobre _buyTokens. Es external para evitar la copia adicional de calldata que implicaría declararla public, y onlyParticipant porque solo los inscritos pueden adquirir más tokens.

sellTokens(uint numTokens) external onlyParticipant

Coste O(1). Comprueba balance del vendedor, quema (burn) los tokens y devuelve el ETH proporcional con un call de bajo nivel. La quema sucede antes del envío de ETH (CEI), por lo que cualquier reentrada encontraría un balance ya descontado.

getERC20() external view returns (address)

Coste O(1). external view por ser un getter consumido únicamente por interfaces externas (los participantes lo usan para invocar approve sobre el ERC20).

addProposal(string calldata, string calldata, uint, address) external onlyParticipant onlyVotingOpen returns (uint)

Coste O(1) gracias a _appendToArray. Comprueba que la dirección sea distinta de cero (pues no es un valor válido) y, mediante IERC165.supportsInterface, valida que el contrato externo implemente realmente IExecutableProposal. Esta comprobación se hace antes de escribir nada en storage, así una propuesta inválida no consume slots. El proposalId se asigna con unchecked porque proposalCount nunca se decrementa y nunca se reinicia entre rondas (esto evita colisiones con rondas previas en el mapping proposals). Según el budget, es decir, si es cero o no, se introduce el ID en pendingFinancingIds o en signalingProposalIds.

cancelProposal(uint proposalId) external onlyVotingOpen

Coste O(1). No lleva onlyParticipant porque el creador podría haberse dado de baja entre la creación y la cancelación; basta con la igualdad msg.sender == p.creator. Cambia el estado a Cancelled y elimina el ID del array correspondiente con swap and pop. No itera sobre votantes ni transfiere tokens (cambio respecto a la versión antigua, ver capítulo 2). Los votantes recuperarán sus tokens cuando llamen a claimRefund.

getPendingProposals, getApprovedProposals, getSignalingProposals

...

external view onlyVotingOpen.

Devuelven el array de la ronda actual. Coste O(n), siendo n el número de propuestas, por la copia en

memoria siempre y cuando se llame desde otro contrato, pero, al ser view, en la práctica, se ejecutará como una `eth_call`, que ejecuta la función localmente en un nodo pero no se incluye en un bloque.

getProposalInfo(uint) external view onlyVotingOpen

Coste $O(1)$. Devuelve los siete campos públicos relevantes de la propuesta. Verifica que la propuesta pertenezca a la ronda actual (descarta IDs antiguos).

stake(uint proposalId, uint numVotes) external onlyParticipant onlyVotingOpen

Coste $O(1)$. El coste cuadrático se calcula en aritmética checked: $\text{tokenCost} = (\text{prevVotes} + \text{numVotes})^2 - \text{prevVotes}^2$. Sigue el orden checks effects interactions de forma estricta: primero validan los require, después se actualizan `voterVotes`, `totalVotes` y `totalTokensStaked`, y solo entonces se invoca `votingToken.transferFrom` para mover los tokens del votante al contrato. Esta operación obliga al votante a haber concedido `approve` previamente sobre el ERC20 (dependencia explícita en los requisitos). Tras la transferencia, si la propuesta es de financiación se invoca `_checkAndExecuteProposal`. Reentradas: la única superficie potencial es el `transferFrom` (controlado, pues es la implementación básica de ERC20) y la llamada interna a `_checkAndExecuteProposal`. Una hipotética reentrada vía el token vería la cuenta del votante ya descontada de votos disponibles y los agregados de la propuesta ya actualizados, así que no podría duplicar voto ni manipular el cálculo del umbral; la reentrada vía la callback del contrato externo queda mitigada por el CEI dentro de `_checkAndExecuteProposal` y por el límite de 100 000 unidades de gas.

withdrawFromProposal(uint proposalId, uint numVotes) external onlyParticipant onlyVotingOpen

Coste $O(1)$. Calcula el reembolso cuadrático mediante $\text{currentVotes}^2 - \text{remaining}^2$ y aplica checks effects interactions de forma estricta: primero los require de validación (votos solicitados positivos, propuesta de la ronda actual, propuesta pendiente y votos suficientes), después la actualización de `voterVotes`, `totalVotes` y `totalTokensStaked`, y por último la transferencia de tokens al llamante con `votingToken.transfer`. Con este orden, una reentrada hipotética desde el ERC20 encontraría `p.voterVotes[msg.sender]` ya reducido a `remaining` y el agregado ya descontado, lo que impide retirar dos veces los mismos votos aunque el token lo permitiera, que no es caso de esta implementación.

claimRefund(uint proposalId) external

Coste $O(1)$. Función nueva del rediseño pull-over-push. Sin modificador de participante: un usuario que se haya dado de baja debe poder seguir reclamando lo suyo. Sólo permite reclamar cuando la propuesta está cancelada (independientemente de la ronda) o cuando la ronda asociada está cerrada y la propuesta sigue Pending. Pone `p.voterVotes[msg.sender]` a cero y descuenta `totalVotes` y `totalTokensStaked` antes del transfer. La doble reclamación queda bloqueada porque el segundo intento ve `votes == 0`.

executeSignaling(uint proposalId) external

Coste $O(1)$ sin contar la llamada externa. Función nueva. Cualquiera puede dispararla una vez cerrada la ronda y siempre que la propuesta sea de signaling y no esté cancelada ni ya ejecutada. Marca `signalingExecuted` en true antes del call al contrato externo (CEI estricto: una reentrada vería el flag activo y abortaría). Reutiliza el límite de 100 000 unidades de gas para acotar el daño que pueda hacer un contrato externo malicioso.

_checkAndExecuteProposal(uint proposalId) internal

Coste $O(1)$. internal porque solo stake la invoca. Aplica un truco algebraico para evitar divisiones: la condición $\text{totalVotes}_i > (0.2 + \text{budget}_i / \text{totalBudget}) * \text{numParticipants} + \text{numPendingProposals}$ se multiplica por $(5 * \text{totalBudget})$ en ambos lados, quedando una comparación exclusivamente con sumas y multiplicaciones sobre enteros. Si la propuesta supera el umbral, actualiza totalBudget sumando el valor en wei de los tokens consumidos y restando el budget concedido, marca la propuesta como Approved, hace swap and pop entre pendingFinancingIds y approvedProposalIds, quema los tokens depositados y, ya con todo el estado finalizado, ejecuta la propuesta externa. La reentrada en stake del mismo proposalId queda bloqueada porque su estado pasa a Approved.

_appendToArray(uint[] storage, uint) internal returns (uint)

Coste $O(1)$. Centraliza el patrón "asignar arrayIndex y push", usado tanto al crear propuesta como al promoverla a aprobada. Reduce código duplicado.

_removeFromArray(uint[] storage, uint) internal

Coste $O(1)$. Implementa swap and pop y propaga el nuevo arrayIndex al elemento desplazado. La operación $\text{arr.length} - 1$ va dentro de unchecked porque el invariante del llamante (la propuesta está en el array) garantiza $\text{arr.length} \geq 1$.

_buyTokens(address, uint) internal

Coste $O(1)$. Calcula tokens, llama a votingToken.mint y devuelve el cambio sobrante por call. El mint se ejecuta antes del envío de ETH (CEI). Una reentrada al addParticipant fallaría en el require de no inscripción, y una reentrada a buyTokens simplemente compraría más tokens legítimamente.

Modificadores y elección entre modifier y require

onlyOwner, onlyParticipant, onlyVotingOpen, onlyVotingClosed se implementan como modificadores porque expresan un control de acceso transversal aplicado en muchas funciones. En cambio, comprobaciones específicas como `require(numVotes > 0, ...)` o `require(_proposalAddr != address(0), ...)` se mantienen inline porque son condiciones particulares de una sola función; encapsularlas en un modifier desplazaría la lógica del flujo principal y obligaría a parametrizar el modifier sin ganar reutilización real. Además, en este caso concreto, el problema que implica parametrizar es que el parámetro sería siempre un Proposal que vive en storage, lo cual generaría una carga por cada modificador, mientras que actualmente se carga una vez.

Vulnerabilidades consideradas y defensa

- Reentrada

Se aplica checks effects interactions en cada punto con llamada externa: closeVoting, sellTokens, addParticipant/buyTokens (devolución de cambio), claimRefund, executeSignaling y _checkAndExecuteProposal. En el caso de _checkAndExecuteProposal, además de CEI, se actualiza el estado del array y del status antes de llamar al contrato externo, de modo que cualquier reentrada vea una propuesta ya finalizada.

- DoS por bucle ilimitado

El rediseño elimina toda iteración sobre votantes o propuestas en `closeVoting` y `cancelProposal`. Las únicas iteraciones residuales son las copias de array en los getters, y los `push/pop` por elemento, que en realidad tienen un coste $O(1)$. Por tanto, ningún atacante puede inflar las estructuras para dejar el sistema bloqueado.

- DoS por gas en callbacks externos

Tanto `_checkAndExecuteProposal` como `executeSignaling` fijan `{gas: 100 000}` en el call al contrato de la propuesta. Una propuesta maliciosa que consuma todo su gas o haga `revert` solo se perjudica a sí misma; el resto de la votación sigue operativa.

- Validación de la interfaz de la propuesta

`addProposal` exige `supportsInterface(type(IExecutableProposal).interfaceId)` sobre la dirección recibida. Si el contrato externo no expone ERC165 correctamente, la propuesta no se llega a registrar y no malgasta storage.

- Manipulación de tokens sin autorización

`VotingToken` solo permite `mint` y `burn` al owner (la instancia de `QuadraticVoting`). El stake usa `transferFrom`, exigiendo `approve` previo del votante. El sistema nunca toca balances ajenos sin una autorización ERC20 explícita.

- Overflow / underflow

Solidity 0.8 hace checked arithmetic por defecto. Los bloques unchecked se usan exclusivamente en tres puntos con invariantes documentados: incrementos de `currentVotingRound`, `participantCount` y `proposalCount` (espacio de `uint256`), y el cálculo `arr.length - 1` en `_removeFromArray` cuando el llamante asegura `length >= 1`.

1.2 Contrato *VotingToken*

Hereda directamente de la implementación ERC20 de OpenZeppelin (ruta `@openzeppelin/contracts/token/ERC20/ERC20.sol`). El nombre es "VotingToken" y el símbolo "VTK". Las decisiones específicas del proyecto son:

`decimals()` devuelve 0 sobrescribiendo el valor por defecto (18). Los votos son enteros (no existe medio voto), así que cualquier representación fraccionaria sería ruido; con cero decimales el `balanceOf` coincide directamente con la cantidad de tokens utilizables.

El campo `owner` es immutable y se asigna al `msg.sender` del constructor, que en la práctica es la instancia de `QuadraticVoting` que lo despliega.

maxSupply (immutable) actúa como cap. La función mint(address, uint) es external onlyOwner y comprueba totalSupply() + amount <= maxSupply antes de invocar _mint heredada.

La función burn(address, uint) también es external onlyOwner y se apoya en _burn de OpenZeppelin. Es la que permite a QuadraticVoting liberar tokens al cerrar votaciones aprobadas y al ejecutar sellTokens.

No se exponen mint ni burn al usuario final: solo se accede a ellos a través de las operaciones de QuadraticVoting (addParticipant, buyTokens, sellTokens, _checkAndExecuteProposal). De este modo, la propiedad de los tokens jamás se altera sin autorización del propietario, ya sea por compra (ETH a cambio), por venta (token a cambio) o por consumo en una propuesta aprobada (donde el votante ya consintió al ejecutar stake).

1.3 IExecutableProposal y TestProposal

IExecutableProposal es una interfaz mínima con una sola función:

```
function executeProposal(uint proposalId, uint numVotes, uint numTokens) external payable;
```

Recibe el identificador, el número total de votos y el número total de tokens depositados en la propuesta. Es payable porque, en el caso de propuestas de financiación, la transacción que la ejecuta lleva el budget en wei. El sistema de votación nunca implementa esta interfaz directamente: lo único que conoce de cada propuesta es la dirección de un contrato externo, lo cual permite al ecosistema desacoplar la lógica de gobernanza de la lógica de ejecución.

TestProposal es el contrato de pruebas que entregamos para verificar el flujo. Hereda de IExecutableProposal y de ERC165 de OpenZeppelin. Define un único event Executed(uint proposalId, uint numVotes, uint numTokens, uint balance) que se emite en cada llamada a executeProposal incluyendo address(this).balance, lo que permite confirmar tanto la ejecución de la callback como la recepción efectiva del ETH presupuestado. Sobreescribe supportsInterface para que devuelva true ante el interfazId de IExecutableProposal, requisito imprescindible para que addProposal acepte la dirección.

2. Pull-over-push

La versión inicial del contrato (conservada como QuadraticVotingOLD.sol) seguía un modelo push: closeVoting iteraba sobre todas las propuestas pendientes y de signaling, y para cada una recorría su lista de votantes para devolver tokens y emitir las llamadas a executeProposal. cancelProposal hacía lo equivalente sobre los votantes de una sola propuesta. Esto producía un consumo de gas lineal sobre el número total de votantes acumulados, vector clásico de denegación de servicio: bastaba con que una propuesta amasara suficientes votantes (algo trivial para un atacante con cuentas baratas) para que closeVoting superase el límite de gas por bloque y el sistema quedara bloqueado, imposibilitando devolver los tokens y abrir nuevas rondas.

La transición al patrón pull over push consistió en delegar al usuario el coste de su propia operación, dejando en el contrato solo trabajo $O(1)$ por transacción. Los cambios concretos respecto a la versión antigua fueron:

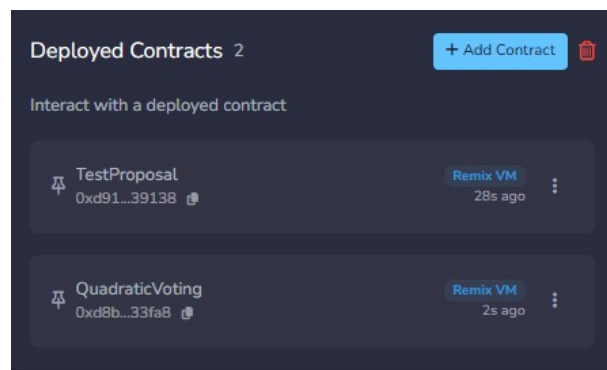
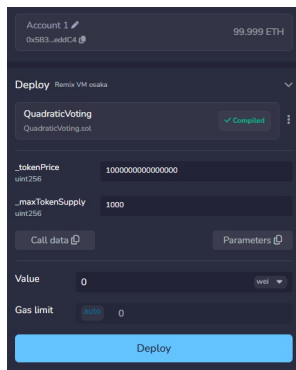
- Eliminación del array `voters[]` y del mapping `hasVoted` en la struct `Proposal`. En la versión vieja sirvieron exclusivamente para iterar en `_returnTokensToVoters`; al desaparecer la iteración, ambos campos quedaron sin uso, así que se borraron junto con la lógica que los mantenía en stake.
- Sustitución del bucle principal de `closeVoting` por dos operaciones: `votingOpen = false` y `roundClosed[currentVotingRound] = true`. Esta segunda asignación es la pieza nueva: actúa como condición que habilita los flujos pull (`claimRefund` y `executeSignaling`) sin riesgo de ejecutarlos durante una ronda viva. La transferencia del presupuesto sobrante al owner se mantiene, por ser $O(1)$.
- Cambio en `cancelProposal`. La versión antigua llamaba a `_returnTokensToVoters`; la nueva solo cambia el estado a `Cancelled` y hace swap and pop sobre el array correspondiente. Los votantes de propuestas canceladas reclaman vía `claimRefund`, también $O(1)$. Esto resuelve el subataque colateral en el que un usuario malicioso podía bloquear `cancelProposal` por sí sola.
- Reorganización de los arrays de propuestas. En la versión antigua eran tres arrays simples (`pendingFinancingIds`, `approvedProposalIds`, `signalingProposalIds`) que se borraban con `delete` al abrir y cerrar votación. En la versión nueva pasan a ser `mapping(uint => uint[])` indexados por la ronda en la que la propuesta fue creada. El motivo es necesario para el patrón pull: los tokens de una ronda cerrada deben poder reclamarse incluso después de que se haya abierto una ronda nueva, lo que obliga a conservar la información de propuestas pasadas. Indexar por ronda permite que cada votación use sus propios slots sin sobrescribir nada y sin pagar el coste del delete masivo.
- Adición de `claimRefund`. Lleva la lógica que antes estaba dentro de `_returnTokensToVoters` al usuario individual. Cubre dos escenarios: propuesta cancelada (cualquier ronda) y propuesta pendiente cuya ronda ya se cerró. Calcula votes^2 a partir de `p.voterVotes[msg.sender]`, pone ese campo a cero, descuenta de los agregados y transfiere los tokens. La imposibilidad de reentrada se garantiza con el `require(votes > 0)`.
- Adición de `executeSignaling`. Reemplaza al bucle de signaling de la versión antigua. Hace pull de la ejecución de propuestas de signaling de rondas cerradas y se protege contra doble disparo con la flag `signalingExecuted`, que se incorporó a la struct `Proposal` precisamente para este flujo. Mantiene el límite de 100 000 unidades de gas por callback.

3. Casos de prueba

1. Despliegue de `TestProposal` y `QuadraticVoting`

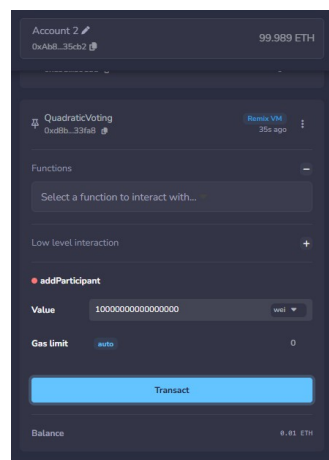
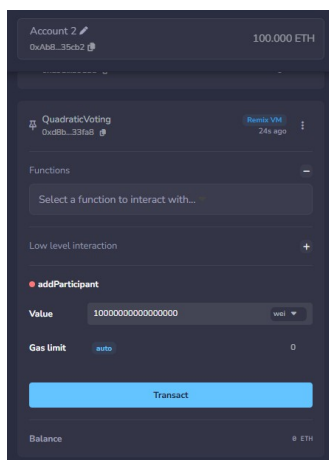
1. Desplegar `TestProposal` desde la cuenta 1 (será reutilizado como receptor de varias propuestas).
2. Desplegar `QuadraticVoting` con `_tokenPrice = 1 000 000 000 000 000` (10^{15} wei = 0.001 ETH por

token) y `_maxTokenSupply` = 1000.

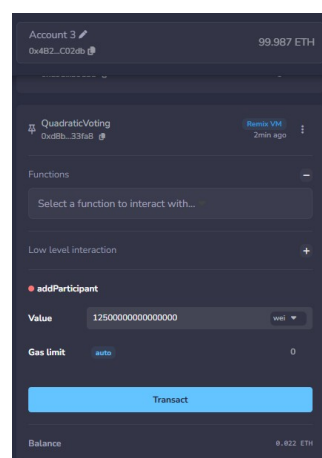
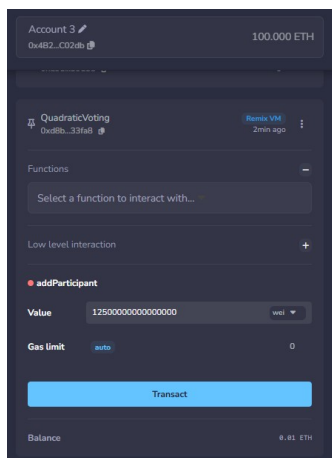


2. Inscripción de participantes antes de abrir la votación

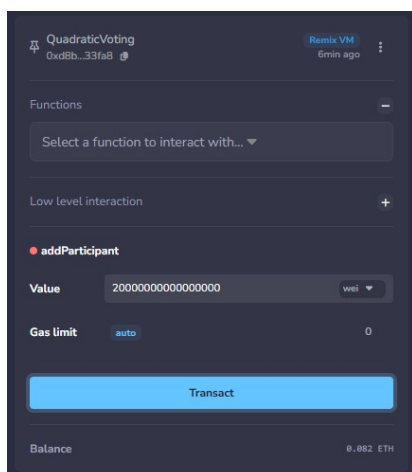
1. Cambiar a la cuenta 2. Llamar a `addParticipant` con value 0.01 ETH (1 000 000 000 0 000 000 wei; compra 10 tokens, sin cambio).



2. Cuenta 3: `addParticipant` con value 0.0125 ETH (1 250 000 000 0 000 000 wei; compra 12 tokens, devuelve 0.0005 ETH de cambio).

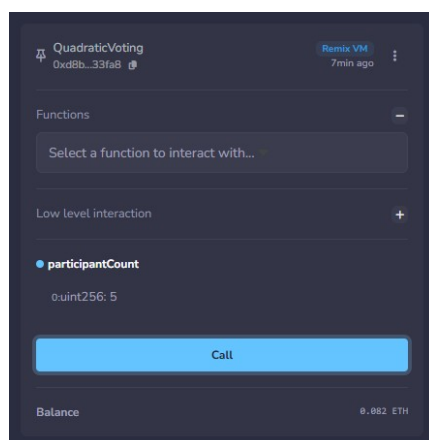


3. Cuenta 4: addParticipant con 0.02 ETH (2 000 000 000 000 000 wei; 20 tokens).
4. Cuenta 5: addParticipant con 0.02 ETH (2 000 000 000 000 000 wei; 20 tokens).
5. Cuenta 6: addParticipant con 0.02 ETH (2 000 000 000 000 000 wei; 20 tokens).

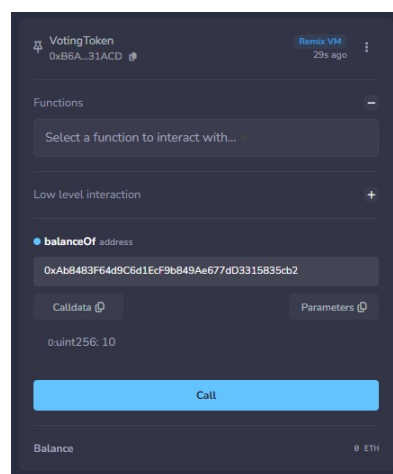
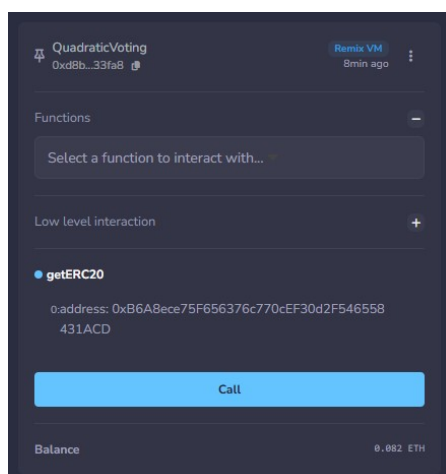


Account 4	99.979 ETH
0x787...cabaB	
Account 5	99.979 ETH
0x617...5E7f2	
Account 6	99.979 ETH
0x17F...8c372	

6. Verificar participantCount == 5.

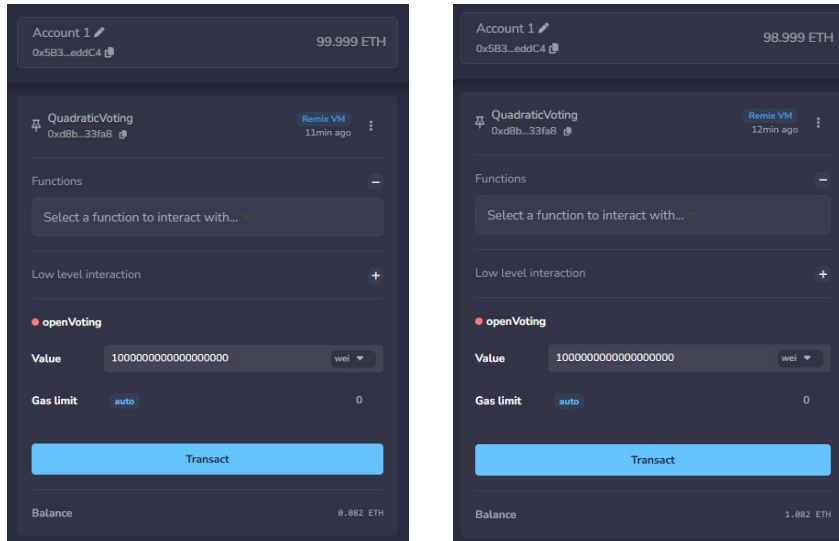


7. Comprobar el balance del votingToken para la cuenta 2 usando getERC20 y balanceOf

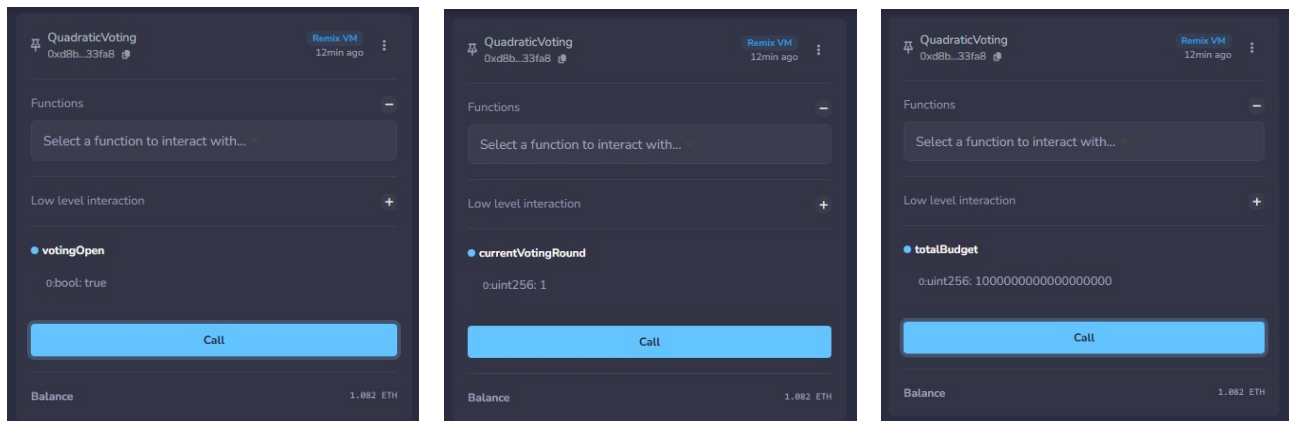


3. Apertura de la votación

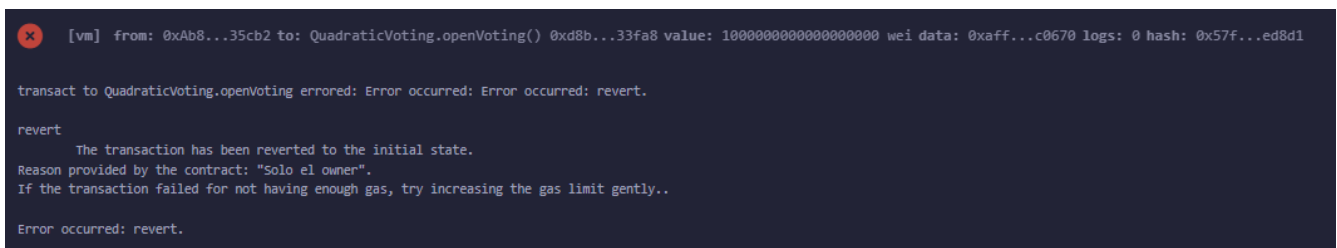
1. Desde la cuenta 1 (owner) llamar a openVoting con value 1 ETH (1 000 000 000 000 000 000 wei).



2. Verificar votingOpen == true, currentVotingRound == 1, totalBudget == 10^18.



3. Comprobar que un intento de openVoting desde la cuenta 2 revierte con "Solo el owner".



4. Creación de propuestas

1. Cuenta 2: addProposal("Construir parque", "Mejora urbana", 5 000 000 000 00 000 000 (0.5 ETH), dirección_TestProposal).

```
decoded input      {
                    "string_title": "Construir parque",
                    "string_description": "Mejora urbana",
                    "uint256_budget": "500000000000000000",
                    "address_proposalAddr": "0xd9145CCE52D386f254917e481e844e9943F39138"
                    }

decoded output      {
                    "0": "uint256: 0"
                    }
```

2. Cuenta 3: addProposal("Documental", "Pieza divulgativa", 2 000 000 000 00 000 000 (0.2 ETH), dirección_TestProposal).

```
decoded input      {
                    "string_title": "Documental",
                    "string_description": "Pieza divulgativa",
                    "uint256_budget": "200000000000000000",
                    "address_proposalAddr": "0xd9145CCE52D386f254917e481e844e9943F39138"
                    }

decoded output      {
                    "0": "uint256: 1"
                    }
```

3. Cuenta 4: addProposal("Encuesta", "Signaling sobre prioridades", 0, dirección_TestProposal).

```
decoded input      {
                    "string_title": "Encuesta",
                    "string_description": "Signaling sobre prioridades",
                    "uint256_budget": "0",
                    "address_proposalAddr": "0xd9145CCE52D386f254917e481e844e9943F39138"
                    }

decoded output      {
                    "0": "uint256: 2"
                    }
```

4. Cuenta 4: addProposal("Otra signaling", "Color del logo", 0, dirección_TestProposal).

```
decoded input      {
                    "string_title": "Otra signaling",
                    "string_description": "Color del logo",
                    "uint256_budget": "0",
                    "address_proposalAddr": "0xd9145CCE52D386f254917e481e844e9943F39138"
                    }

decoded output      {
                    "0": "uint256: 3"
                    }
```

5. Llamar a getPendingProposals (debería devolver [0, 1]) y getSignalingProposals ([2, 3]).

```
decoded output      {
                    "0": "uint256[]: 0,1"
                    }

decoded output      {
                    "0": "uint256[]: 2,3"
                    }
```

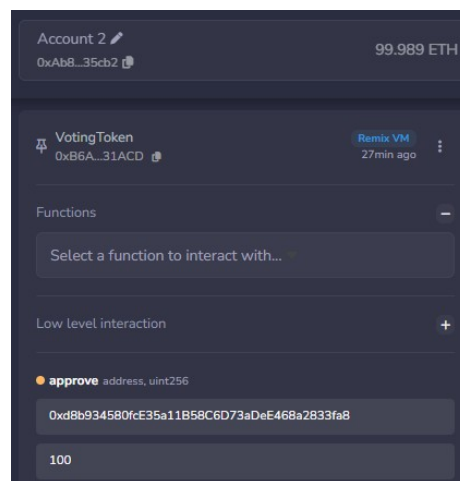
6. Llamar a `getProposalInfo(0)` y comprobar los siete campos.

```
decoded input      {
                    "uint256 proposalId": "0"
                  }

decoded output     {
                    "0": "string: title Construir parque",
                    "1": "string: description Mejora urbana",
                    "2": "uint256: budget 5000000000000000",
                    "3": "uint256: totalVotes 0",
                    "4": "uint8: status 0",
                    "5": "address: proposalAddress 0xd9145CCE52D386f254917e481eB44e9943F39138",
                    "6": "address: creator 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2"
                  }
```

5. Voto cuadrático y aprobación automática

1. Desde la cuenta 2: aprobar al contrato `QuadraticVoting` el gasto de tokens. Llamar a `approve(direccion_QuadraticVoting, 100)`. Repetir desde las cuentas 3, 4, 5 y 6.



2. Cuenta 2: `stake(0, 2)`. Coste 4 tokens. Comprobar `balanceOf` antes y después (10 -> 6).

```
decoded input      {
                    "address account": "0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2"
                  }

decoded output     {
                    "0": "uint256: 10"
                  }
```

```
decoded input      {
                    "address account": "0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2"
                  }

decoded output     {
                    "0": "uint256: 6"
                  }
```

3. Cuenta 3: stake(0, 3). Coste 9 tokens (12 -> 3).

```
decoded input      {
                    "address account": "0x4B209938c481177ec7E8f571ceCaE8A9e22C02db"
                    }
decoded output      {
                    "0": "uint256: 12"
                    }
```

```
decoded input      {
                    "address account": "0x4B209938c481177ec7E8f571ceCaE8A9e22C02db"
                    }
decoded output      {
                    "0": "uint256: 3"
                    }
```

p.totalVotes = 5
totalBudget = 1 000 000 000 000 000 000 wei (1 ETH)
p.budget = 5 000 000 000 000 000 000 wei (0.5 ETH)
participantCount = 5
pendingFinancingIds[currentVotingRound].length = 2

5 * 5 * 1 000 000 000 000 000 000 wei (1 ETH)
>
(1 000 000 000 000 000 000 wei (1 ETH) + 5 * 5 000 000 000 000 000 000 wei (0.5 ETH)) * 5
+
2 * 5 * 1 000 000 000 000 000 000 wei (1 ETH)

2 500 000 000 000 000 000
>
2 750 000 000 000 000 000

Por tanto, aún no cruza el umbral.

4. Cuenta 4: stake(0, 3). Coste 9 tokens (20 -> 11).

```
decoded input      {
                    "address account": "0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB"
                    }
decoded output      {
                    "0": "uint256: 20"
                    }
```

```
decoded input      {
                    "address account": "0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB"
                    }
decoded output      {
                    "0": "uint256: 11"
                    }
```

p.totalVotes = 8

```
totalBudget = 1 000 000 000 000 000 000 wei (1 ETH)
p.budget = 5 000 000 000 000 000 000 wei (0.5 ETH)
participantCount = 5
pendingFinancingIds[currentVotingRound].length = 2
```

```
8 * 5 * 1 000 000 000 000 000 000 wei (1 ETH)
>
(1 000 000 000 000 000 000 wei (1 ETH) + 5 * 5 000 000 000 000 000 000 wei (0.5 ETH)) * 5
+
2 * 5 * 1 000 000 000 000 000 000 wei (1 ETH)

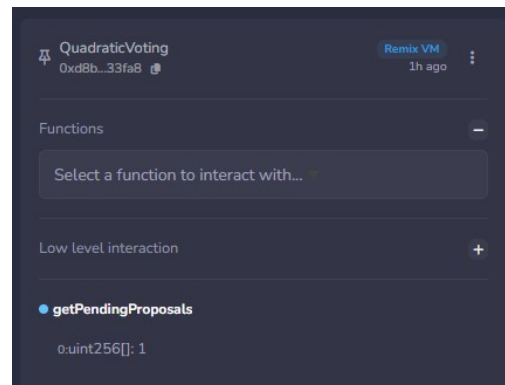
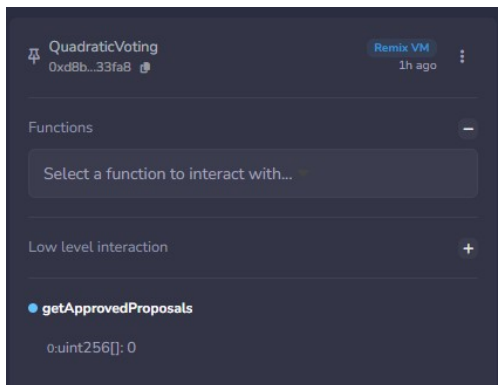
4 000 000 000 000 000 000
>
2 750 000 000 000 000 000
```

Por tanto, ahora sí cruza el umbral y se llama a executeProposal. Lo cual podemos comprobar con el evento emitido

```
Event: Executed(uint256,uint256,uint256,uint256)

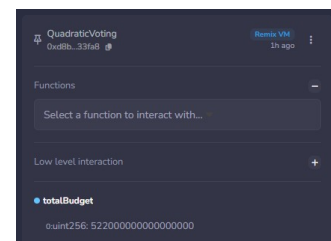
proposalId: 0
numVotes: 8
numTokens: 22
balance: 500000000000000000 (0.5000 ETH)
```

Y con los arrays de Proposal:



También, podemos comprobar que los 22 tokens se han quemado y se ha añadido al totalBudget $(1\,000\,000\,000\,000\,000\,000 + (22 * 1\,000\,000\,000\,000\,000) - 5\,000\,000\,000\,000\,000\,000 == 5\,220\,000\,000\,000\,000\,000)$

```
{
  "from": "0xB6A8ece75F656376c770cEF30d2F546558431ACD",
  "topic": "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
  "event": "Transfer",
  "args": {
    "0": "0xd8b934580fcE35a11858C6D73aDeE468a2833fa8",
    "1": "0x0000000000000000000000000000000000000000000000000000000000000000",
    "2": "22"
  }
}
```



6. Retirada parcial de votos

1. Cuenta 5: stake(1, 3) sobre la propuesta 1

```
decoded input      {
                    "address account": "0x617f2E2fD72FD9D5503197092aC168c91465E7f2"
                    }
decoded output      {
                    "0": "uint256: 20"
                    }
```

```
decoded input      {
                    "address account": "0x617f2E2fD72FD9D5503197092aC168c91465E7f2"
                    }
decoded output      {
                    "0": "uint256: 11"
                    }
```

p.totalVotes = 3

totalBudget = 5 220 000 000 00 000 000 wei (0.522 ETH)

p.budget = 2 000 000 000 00 000 000 wei (0.2 ETH)

participantCount = 5

pendingFinancingIds[currentVotingRound].length = 1

$3 * 5 * 5\,220\,000\,000\,00\,000\,000$ wei (0.522 ETH)

>

$(5\,220\,000\,000\,00\,000\,000$ wei (0.522 ETH) + $5 * 2\,000\,000\,000\,00\,000\,000$ wei (0.2 ETH)) * 5

+

$1 * 5 * 5\,220\,000\,000\,00\,000\,000$ wei (0.522 ETH)

7 830 000 000 000 000 000

>

1 022 000 000 0 000 000 000

Por tanto, no se cruza el umbral.

2. Cuenta 5: withdrawFromProposal(1, 2). Tokens devueltos: $3^2 - 1^2 = 8$. Comprobar balanceOf y getProposalInfo(1).totalVotes.

```
decoded input      {
                    "address account": "0x617f2E2fD72FD9D5503197092aC168c91465E7f2"
                    }
decoded output      {
                    "0": "uint256: 19"
                    }
```

```
decoded input      {
                    "uint256 proposalId": "1"
                    }
decoded output      {
                    "0": "string: title Documental",
                    "1": "string: description Pieza divulgativa",
                    "2": "uint256: budget 2000000000000000000",
                    "3": "uint256: totalVotes 1",
                    "4": "uint8: status 0",
                    "5": "address: proposalAddress 0xd9145CC520386f254917e481e844e9943f39138",
                    "6": "address: creator 0x4b209938c481177ec7E8f571ceCa8A9e22C02db"
                    }
```

7. Cancelación

1. Cuenta 5: addProposal("A cancelar", "Prueba", 1 000 000 000 00 000 000 (0.1 ETH), direccion_TestProposal).

```
decoded input      {
                    "string _title": "A cancelar",
                    "string _description": "Prueba",
                    "uint256 _budget": "100000000000000000",
                    "address _proposalAddr": "0xd9145CCE52D386f254917e481eB44e9943F39138"
                    }
decoded output      {
                    "0": "uint256: 4"
                    }
```

2. Cuenta 6: stake(4, 2) (coste 4 tokens).

```
decoded input      {
                    "address account": "0x17F6AD8Ef982297579C203069C1DbfFE4348c372"
                    }
decoded output      {
                    "0": "uint256: 20"
                    }
```

```
decoded input      {
                    "address account": "0x17F6AD8Ef982297579C203069C1DbfFE4348c372"
                    }
decoded output      {
                    "0": "uint256: 16"
                    }
```

p.totalVotes = 2

totalBudget = 5 220 000 000 00 000 000 wei (0.522 ETH)

p.budget = 1 000 000 000 00 000 000 (0.1 ETH)

participantCount = 5

pendingFinancingIds[currentVotingRound].length = 2

2 * 5 * 5 220 000 000 00 000 000 wei (0.522 ETH)

>

(5 220 000 000 00 000 000 wei (0.522 ETH) + 5 * 1 000 000 000 00 000 000 (0.1 ETH)) * 5

+

2 * 5 * 5 220 000 000 00 000 000 wei (0.522 ETH)

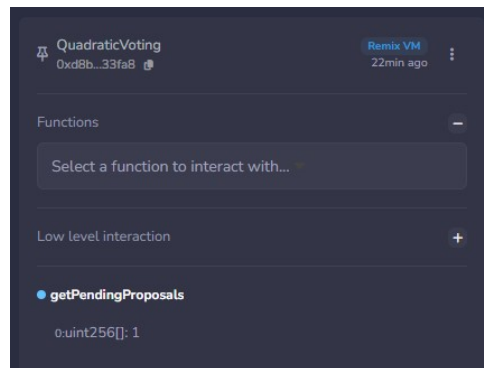
5 220 000 000 00 000 000

>

1033 000 000 0 000 000 000

Por tanto, no se cruza el umbral.

3. Cuenta 5: cancelProposal(4).

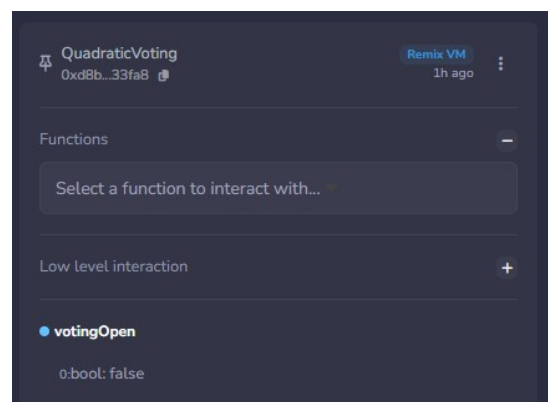
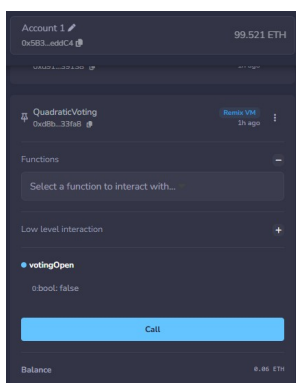


4. Cuenta 5: claimRefund(4) (debería devolver 4 tokens y permitir verificar el incremento del balance).



8. Cierre de la votación y flujos pull

1. Desde la cuenta 1: closeVoting. Verificar que el balance de la cuenta 0 aumenta con el sobrante del totalBudget y que votingOpen pasa a false. Queda en QuadraticVoting lo referente a tokens.



2. Llamar a `executeSignaling(2)` desde cualquier cuenta. Verificar el event `Executed` para la propuesta de signaling 2.

```
Event: Executed(uint256,uint256,uint256,uint256)

proposalId: 2
numVotes: 0
numTokens: 0
balance: 5000000000000000000 (0.5000 ETH)
```

3. Llamar a `executeSignaling(3)` y comprobar que un segundo intento revierte con "Ya ejecutada".

```
Event: Executed(uint256,uint256,uint256,uint256)

proposalId: 3
numVotes: 0
numTokens: 0
balance: 5000000000000000000 (0.5000 ETH)
```

```
[vm] from: 0x583...eddC4 to: QuadraticVoting.executeSignaling(uint256) 0xd8b...33fa8 value: 0 wei data: 0xda6...00003 logs: 0 hash: 0xae3...0e55e

transact to QuadraticVoting.executeSignaling errored: Error occurred: Error occurred: revert.

revert
  The transaction has been reverted to the initial state.
Reason provided by the contract: "Ya ejecutada".
If the transaction failed for not having enough gas, try increasing the gas limit gently..

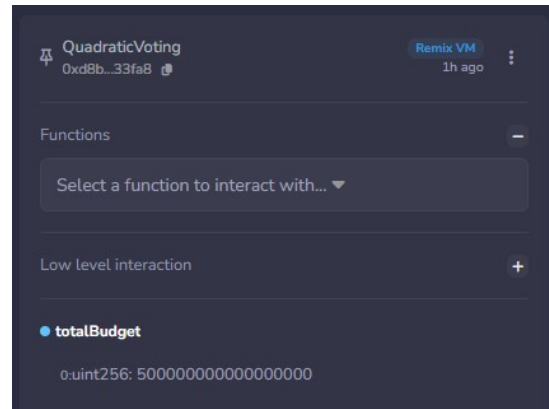
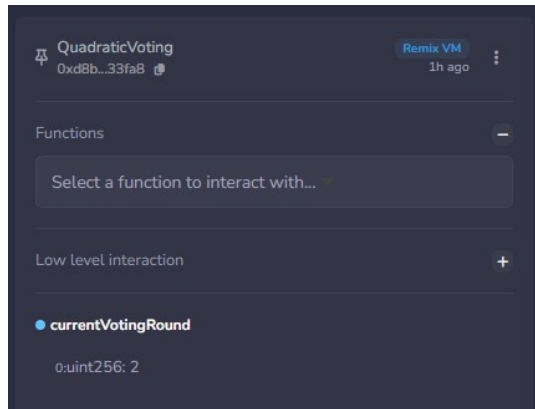
Error occurred: revert.
```

4. Cuentas que hubieran votado propuestas no aprobadas: Cuenta 5 `claimRefund(1)`. Confirmar la devolución de tokens correspondiente.

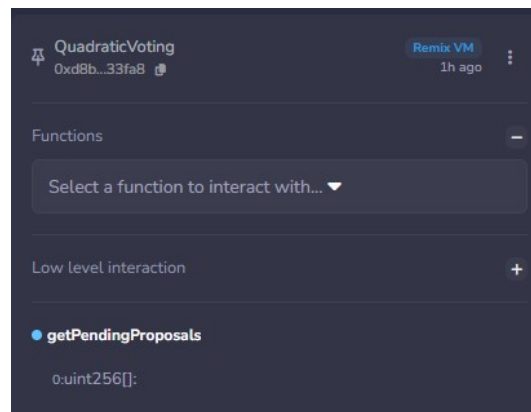
```
logs
[
  {
    "from": "0xB6A8ece75F656376c770cEF30d2F546558431ACD",
    "topic": "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
    "event": "Transfer",
    "args": {
      "0": "0xd8b934580fce35a11858C6D73aDeE468a2833fa8",
      "1": "0x617F2E2fD72FD9D5503197092aC168c91465E7f2",
      "2": "1"
    }
  }
]
```

9. Reapertura

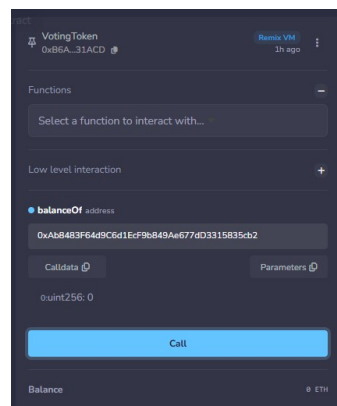
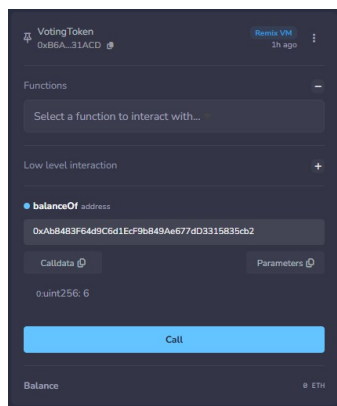
1. Desde la cuenta 1: openVoting con value 0.5 ETH (5 000 000 000 000 000 wei). Verificar `currentVotingRound == 2` y `totalBudget` actualizado.

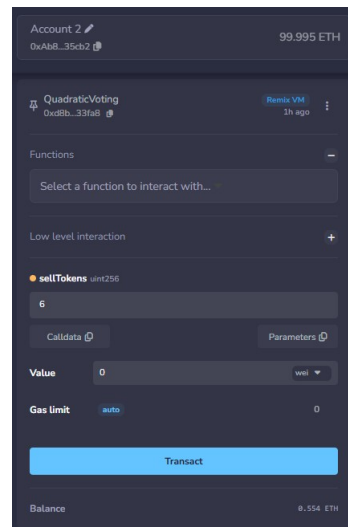
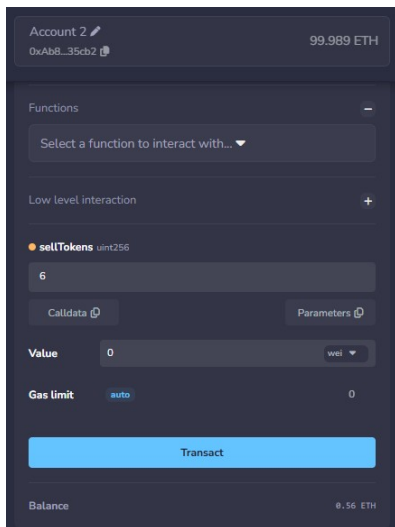


2. Comprobar que `getPendingProposals` devuelve un array vacío (los IDs de la ronda anterior siguen accesibles vía `claimRefund` pero no asoman en la nueva ronda).



3. Las cuentas que conserven tokens pueden votar nuevas propuestas en esta ronda; las que prefieran salir, `sellTokens`.





10. Comprobaciones de seguridad

1. Desde una cuenta no participante (por ejemplo la 9): intentar stake. Debe revertir con "No es participante".

```
[vm] from: 0x1aE...E454C to: QuadraticVoting.stake(uint256,uint256) 0xd8b...33fa8 value: 0 wei data: 0x7b0...00002 logs: 0 hash: 0x9e8...e19b2

transact to QuadraticVoting.stake errored: Error occurred: Error occurred: revert.

revert
  The transaction has been reverted to the initial state.
Reason provided by the contract: "No es participante".
If the transaction failed for not having enough gas, try increasing the gas limit gently..

Error occurred: revert.
```

2. Desde la cuenta 1: intentar addProposal sin ser participante. Debe revertir con el mismo motivo.

```
[vm] from: 0x583...eddC4 to: QuadraticVoting.addProposal(string,string,uint256,address) 0xd8b...33fa8 value: 0 wei data: 0x9ab...00000 logs: 0 hash: 0x3f2...e55ac

transact to QuadraticVoting.addProposal errored: Error occurred: Error occurred: revert.

revert
  The transaction has been reverted to the initial state.
Reason provided by the contract: "No es participante".
If the transaction failed for not having enough gas, try increasing the gas limit gently..

Error occurred: revert.
```

3. Intentar addProposal con la dirección de QuadraticVoting (que no implementa IExecutableProposal). Revierte con "No implementa IExecutableProposal".

```
[vm] from: 0xAB8...35cb2 to: QuadraticVoting.addProposal(string,string,uint256,address) 0xd8b...33fa8 value: 0 wei data: 0x9ab...00000 logs: 0 hash: 0x14a...f12f6

transact to QuadraticVoting.addProposal errored: Error occurred: Error occurred: revert.

revert
  The transaction has been reverted to the initial state.
Reason provided by the contract: "No implementa IExecutableProposal".
If the transaction failed for not having enough gas, try increasing the gas limit gently..

Error occurred: revert.
```